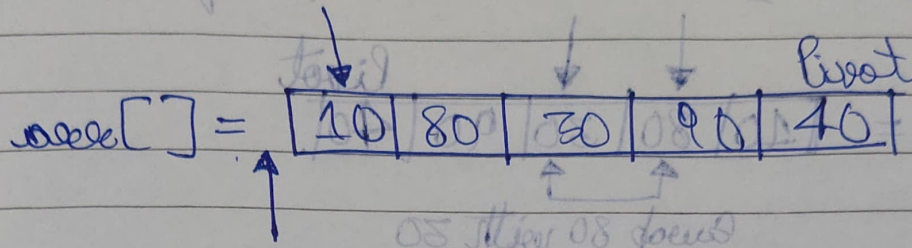


• Working of Partition Algorithm

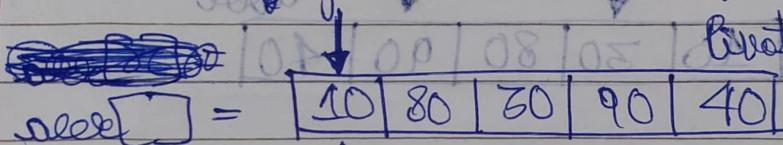
We start from the leftmost element and keep track of the index of a smaller (or equal) element as i . While travelling, if we find a smaller element, we swap the current element with $arr[i]$. Otherwise, we ignore the current element.

→
Start
List

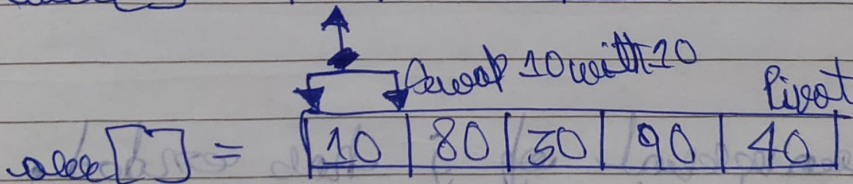
Pivot Selection: The last element $arr[4] = 40$ is chosen as the pivot.
Initial pointers: $i = -1$ and $j = 0$.



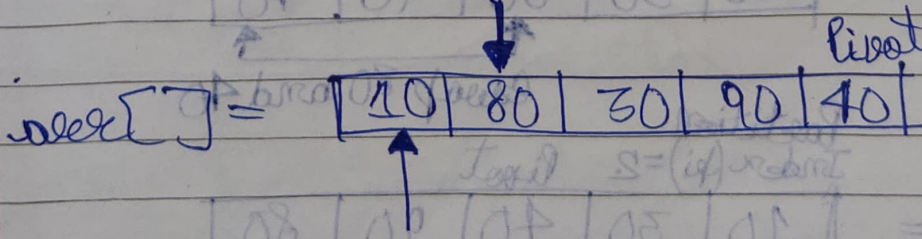
Since, $arr[j] < pivot$ ($10 < 40$)
Increment i to 0 and swap $arr[i]$ with $arr[j]$. Increment j by 1.



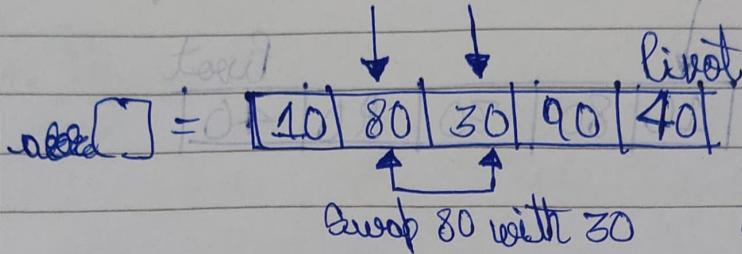
#Every increment i and then swap its smallest value



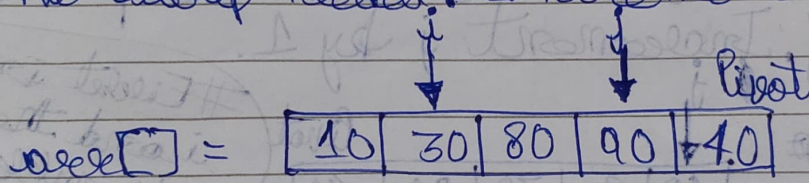
Since, $arr[j] > pivot$ ($80 > 40$)
No swap needed. Increment j by 1



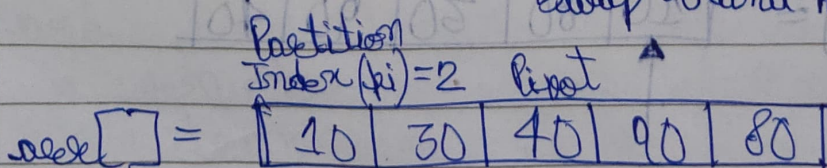
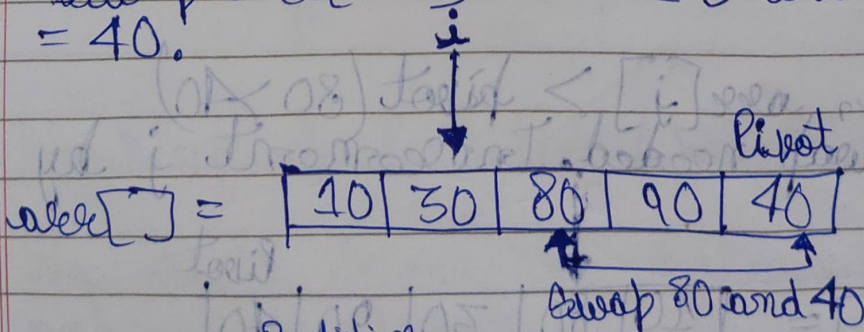
Since, $arr[j] < pivot$ ($30 < 40$)
 Increment i by 1 and swap $arr[i]$
 with $arr[j]$. Increment j by 1.



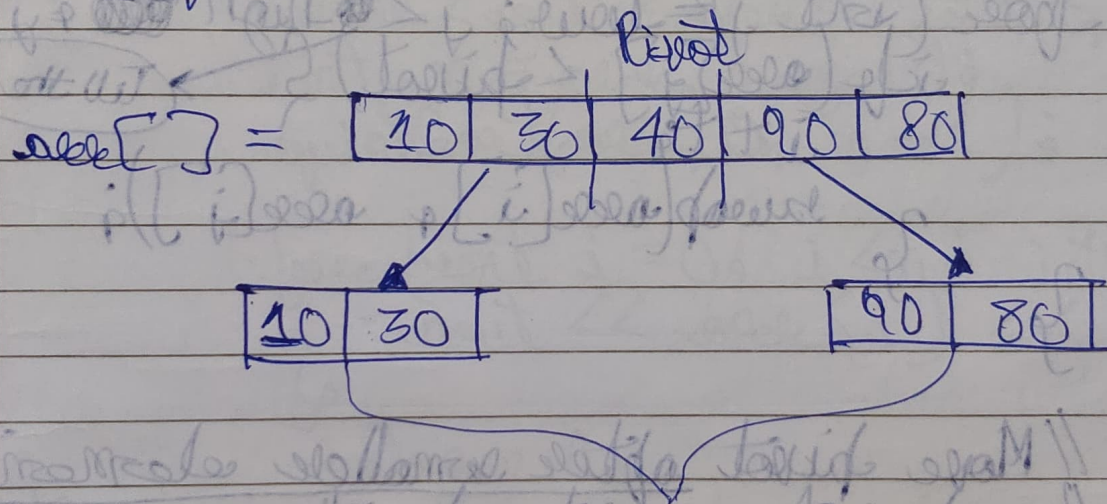
Since, $arr[j] > pivot$ ($90 > 40$)
 No swap needed. Increment j by 1.



Since traversal of j has ended. Move
 more pivot to its correct position,
 swap $arr[i+1] = arr[2]$ with $arr[4]$
 $= 40$.



We apply the same method recursively to the smaller sub-arrays on the left and right of the pivot. Each time, we select new pivots and partition the arrays again. This process continues until only one element is left, which is always sorted. Once every element is in its sorted position, the entire array is sorted.



Recursively call quickSort for left portion (from low to $pi-1$) and right portion (from $pi+1$ to high) of the array to sort the elements around pivot.